Building a Custom Multi-Platform FPGA-SoC Linux Kernel on WSL2

A practical guide from zero to a cross-compiled kernel (ikwzm reference, Linux 6.12.100)

1. Objective and reference

This guide builds a custom Linux kernel for FPGA-SoC platforms from WSL2 by cross-compiling a vanilla kernel with the ikwzm patch set. The result is a kernel with FPGA Manager, FPGA Bridge, FPGA Region and Device Tree Overlay support, plus decompiled Device Trees to use as blueprints. It is the starting point for Agilex 7, Agilex 9 and Jetson kernels (see Section 9).

Repository	`https://github.com/ikwzm/FPGA-SoC-Linux-Kernel-6.12` (maintainer: KAWAZOME Ichiro)
Tag used	`6.12.100-armv7-fpga-1`
Hardware	AMD/Xilinx Zynq-7000 (ZYBO, ZYBO-Z7, PYNQ-Z1); Intel/Altera Cyclone V (DE0-Nano-SoC, DE10-Nano)
Features	Single image for Xilinx and Intel SoCs; `CONFIG_OF_OVERLAY`, `CONFIG_FPGA_MGR`, `CONFIG_FPGA_BRIDGE`, `CONFIG_FPGA_REGION`; USB ChipIdea patch for PYNQ-Z1

2. Prepare the WSL2 environment

Step 1: Update the system and install build tools

TERMINAL

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential bc bison flex libssl-dev libncurses5-dev \
    u-boot-tools device-tree-compiler wget git curl patch debhelper cpio \
    gcc-arm-linux-gnueabihf binutils-arm-linux-gnueabihf
```

- `gcc-arm-linux-gnueabihf`, `binutils-arm-linux-gnueabihf`: 32-bit ARM cross-compiler and linker/assembler.
- `device-tree-compiler`: provides `dtc`. `u-boot-tools`: U-Boot image tools.
- `debhelper`, `cpio`: required by the `deb-pkg` target (the build aborts without them).
- `bc`, `bison`, `flex`, `libssl-dev`, `libncurses5-dev`: kernel build and `menuconfig` dependencies.

Verify the cross-compiler:

TERMINAL

```
arm-linux-gnueabihf-gcc --version
```

EXPECTED OUTPUT

```
arm-linux-gnueabihf-gcc (Ubuntu ...) ...    # version string varies by Ubuntu release
```

Step 2: Work in the native Linux filesystem

TERMINAL

```
cd ~
pwd
```

> Expected: `/home/YOUR_USERNAME`. If you see `/mnt/c/...` you are on the Windows mount; go back to `~`. Building there is slow and breaks file permissions.

3. Fetch the sources

Step 3: Clone the ikwzm reference repository

TERMINAL

```
git clone --branch 6.12.100-armv7-fpga-1 --depth 1 \
    https://github.com/ikwzm/FPGA-SoC-Linux-Kernel-6.12.git ikwzm-ref
```

The "detached HEAD" warning is normal: Git checked out a tag, not a branch. Confirm the contents:

TERMINAL

```
cd ~/ikwzm-ref
ls -l
```

You must see `files/` (defconfig), `patches/` (three `.diff` files), `doc/`, `devicetrees/`, `scripts/`, and the pre-built `.deb` packages and `vmlinuz-6.12.100-armv7-fpga-1`.

Step 4: Clone the vanilla Linux 6.12.100 source and create a build branch

TERMINAL

```
cd ~
git clone --depth 1 -b v6.12.100 \
    git://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git \
    linux-6.12.100-armv7-fpga
cd ~/linux-6.12.100-armv7-fpga
git describe --tags
git checkout -b custom-fpga-build
```

EXPECTED OUTPUT

```
v6.12.100
Switched to a new branch 'custom-fpga-build'
```

> The download is about 300 to 500 MB. If `git.kernel.org` is unreachable, use the HTTPS URL `https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git` instead.

4. Patch and configure

Step 5: Apply the three patches (run inside the kernel directory, in this order)

TERMINAL

```
patch -p1 < ../ikwzm-ref/patches/linux-6.12.100-armv7-fpga.diff
patch -p1 < ../ikwzm-ref/patches/linux-6.12.100-armv7-fpga-usb-ulpi.diff
patch -p1 < ../ikwzm-ref/patches/linux-6.12.100-armv7-fpga-builddeb.diff
```

Each patch prints `patching file ...` lines and must report no `FAILED` hunks.

Step 6: Install the custom defconfig and commit a checkpoint

TERMINAL

```
cp ../ikwzm-ref/files/armv7_fpga_defconfig arch/arm/configs/
ls -l arch/arm/configs/armv7_fpga_defconfig
git add --all
git commit -m "Apply ikwzm FPGA-SoC patches and custom defconfig"
echo 0 > .version
```

Step 7: Confirm the FPGA subsystem is enabled

TERMINAL

```
grep "CONFIG_FPGA" arch/arm/configs/armv7_fpga_defconfig | head -n 5
```

EXPECTED OUTPUT

```
CONFIG_FPGA=y
CONFIG_FPGA_MGR_SOCFPGA=y
CONFIG_FPGA_MGR_ZYNQ_FPGA=y
CONFIG_FPGA_BRIDGE=y
CONFIG_FPGA_REGION=y
```

`=y` means the driver is built into the kernel binary (`=m` would be a loadable module). Built-in is preferred for boot-time FPGA management.

5. Cross-compile the kernel

Step 8: Set the cross-compilation environment

TERMINAL

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
export DTC_FLAGS=--symbols
echo $ARCH $CROSS_COMPILE
```

> These variables last only for the current terminal session; set them again after reopening the terminal. `DTC_FLAGS=--symbols` makes `dtc` emit symbol information, which Device Tree Overlays need at runtime.

Step 9: Generate the config and build

TERMINAL

```
make armv7_fpga_defconfig
make clean
make -j$(nproc) deb-pkg
```

The build prints thousands of lines and ends by creating the `linux-image`, `linux-headers` and `linux-libc-dev` packages. Do not interrupt it. If the machine runs out of RAM, use `make -j2 deb-pkg`.

6. Verify the build artifacts

Step 10: Locate packages, kernel image and Device Tree blobs

TERMINAL

```
cd ~
ls -lh *.deb
find ~/linux-6.12.100-armv7-fpga -name "zImage" -o -name "vmlinuz*" 2>/dev/null
find ~/linux-6.12.100-armv7-fpga -name "*.dtb" | grep -v "empty_root" | head -n 10
```

A `-dirty` suffix in the version string (e.g. `6.12.100-armv7-fpga-g52a355b23cd2-dirty`) only means uncommitted changes existed at build time. It is harmless.

Artifact	Location	Purpose
`linux-image-*.deb`	`~/`	Installable kernel and modules
`linux-headers-*.deb`	`~/`	Headers for out-of-tree modules
`linux-libc-dev_*.deb`	`~/`	C library development headers
`zImage`	`<kernel dir>/arch/arm/boot/`	Compressed kernel binary
`*.dtb`	`<kernel dir>/arch/arm/boot/dts/`	Compiled Device Trees
`.config`	`<kernel dir>/`	Active kernel configuration

7. Analyse the Device Tree

Step 11: Decompile a DTB into readable source

TERMINAL

```
cd ~/linux-6.12.100-armv7-fpga/arch/arm/boot/dts/intel/socfpga
dtc -I dtb -O dts -o ~/intel-soc-reference.dts socfpga_cyclone5_de0_nano_soc.dtb
head -n 60 ~/intel-soc-reference.dts
```

Warnings such as `simple_bus_reg ... missing or empty reg/ranges property` are normal when decompiling and can be ignored.

Step 12: Extract the FPGA bridge, manager and region nodes

TERMINAL

```
grep -A 10 "fpga-mgr\|fpga-bridge\|fpga-region" ~/intel-soc-reference.dts
```

EXPECTED OUTPUT

```
fpga_bridge@ff500000 { compatible = "altr,socfpga-hps2fpga-bridge"; ... };
fpgamgr@ff706000     { compatible = "altr,socfpga-fpga-mgr"; ... };
fpga-region0 {
    compatible = "fpga-region";
    fpga-mgr = <0x24>;
    fpga-bridges = <0x25 0x26 0x27 0x28>;
};
```

> These addresses (e.g. `0xff500000`) are specific to Cyclone V. For Agilex 7/9 take them from the Intel Agilex Hard Processor System Technical Reference Manual.

8. Final verification and troubleshooting

Step 13: Confirm both FPGA Manager drivers are built in

TERMINAL

```
grep "CONFIG_FPGA_MGR" ~/linux-6.12.100-armv7-fpga/.config
```

EXPECTED OUTPUT

```
CONFIG_FPGA_MGR_SOCFPGA=y
# CONFIG_FPGA_MGR_SOCFPGA_A10 is not set
CONFIG_FPGA_MGR_ZYNQ_FPGA=y
...
```

`CONFIG_FPGA_MGR_SOCFPGA=y` (Intel) and `CONFIG_FPGA_MGR_ZYNQ_FPGA=y` (Xilinx) together confirm that one kernel image manages FPGA hardware on both vendors' platforms.

Common errors and fixes

Error	Fix
`unmet build dependencies: debhelper-compat ... cpio`	`sudo apt install -y debhelper cpio`, then rebuild
`can't find file to patch` / `Hunk #X FAILED`	Run from inside the kernel directory; check `git describe --tags` prints `v6.12.100`
`Can't find default configuration ...armv7_fpga_defconfig`	Defconfig not copied; repeat Step 6
`openssl/ssl.h: No such file or directory`	`sudo apt install -y libssl-dev`
`Killed` during compilation	Out of RAM; use `make -j2 deb-pkg`
`No space left on device`	Free disk space or move the build to a larger partition
`Unable to locate package gcc-arm-linux-gnueabihf`	`sudo add-apt-repository universe`, then `sudo apt update`
`unable to connect to git.kernel.org`	Use the HTTPS clone URL (Step 4)

9. Adapting this workflow to Agilex 7/9 and Jetson

The workflow above targets 32-bit ARMv7. For 64-bit ARM targets change the toolchain, base configuration and Device Trees:

TERMINAL

```
sudo apt install -y gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu
export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-
```

Target	Base configuration	Device Tree directory
Intel Agilex 7/9	Generic arm64 `defconfig`; vendor tree: `altera-fpga/linux-socfpga`	`arch/arm64/boot/dts/intel/`
NVIDIA Jetson	NVIDIA L4T kernel (or OE4T `meta-tegra`); arm64 `defconfig` as mainline baseline	`arch/arm64/boot/dts/nvidia/`

> `socfpga_defconfig` and `tegra_defconfig` exist for 32-bit ARM only; on arm64 start from the generic `defconfig` or the vendor tree. Jetson GPU/DLA support depends on NVIDIA's out-of-tree modules, so Jetson realistically keeps its own kernel image. Verify these paths in the exact tree you use.

After loading the base config, run `make menuconfig` and confirm these options:

TERMINAL

```
CONFIG_FPGA=y
CONFIG_FPGA_MGR=y
CONFIG_FPGA_BRIDGE=y
CONFIG_FPGA_REGION=y
CONFIG_OF_OVERLAY=y
CONFIG_OF_CONFIGFS=y
```

Device Tree for Agilex: use `intel-soc-reference.dts` as a template, then (1) set the `compatible` string for your board, (2) replace every address with values from the Agilex HPS Technical Reference Manual, and (3) update the CPU nodes to the Cortex-A53 (or later) cores of your device.